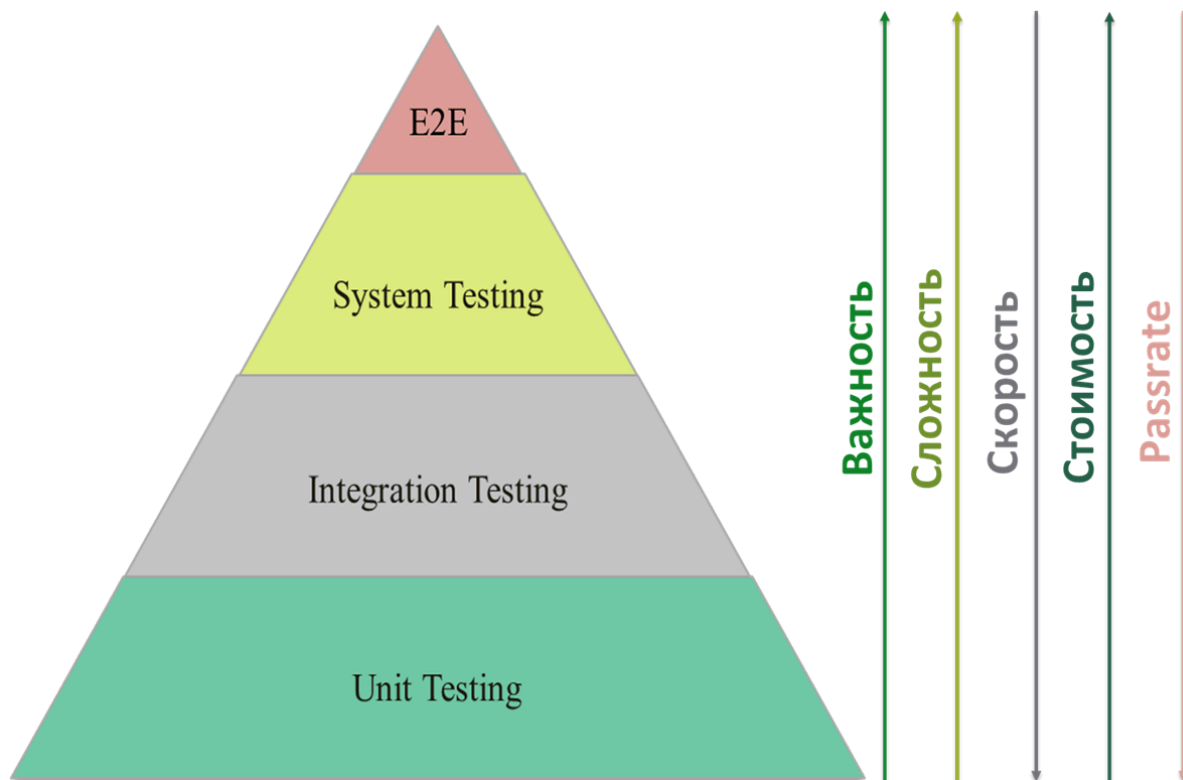


Лекция 2 Классификация тестирования

По уровням тестирования

Пирамида тестирования

«Пирамида тестов» - метафора, которая означает группировку динамических тестов программного обеспечения по разным уровням. Она также дает представление, какое количество тестов должно быть в каждой из этих групп. Основной принцип разделения уровней - тест должен быть на том же уровне, что и тестируемый объект. В тесте более высокого уровня вы не тестируете всю условную логику и пограничные случаи, которые уже покрыты тестами более низкого уровня.



Уровни тестирования:

- Unit/component/program/module testing - тестируется минимально-атомарный модуль программы, чаще всего это одна функция или метод. Таких тестов должно быть больше всего;
- Integration testing - несколько модулей программы тестируются вместе;

- System testing - вся программа тестируется полностью;
- Acceptance testing - программа принимается заказчиком на соответствие заявленным требованиям либо тестировщики проходят end-to-end сценарии с точки зрения пользователя;

Источник:

https://vladisla-veremeev.gitbook.io/qa_bible/vidy-metody-urovni-testirovaniya/piramida-urovni-testirovaniya-test-pyramid-testing-levels

Примечание

На картинке выше вы видите пирамиду тестирования, где уровень UAT заменен на E2E тесты. В контексте данной пирамиды это одно и то же.

Если копнуть глубже, то **end-to-end тестирование, или сквозное тестирование** - это вид тестирования, используемый для проверки программного обеспечения от начала до конца, а также его интеграцию с внешними интерфейсами.

На примере интернет магазина, это одна объемная проверка, которая включает в себя все шаги пользователя, начиная от регистрации в системе и заканчивая покупкой товара в магазине.

Также вы можете обратить внимание на увеличение/уменьшение важности, сложности, скорости, стоимости, уровня прохождения на этих уровнях.

Модульное (юнит, компонентное) тестирование

Компонентное тестирование (component testing) - уровень тестирования отдельных аппаратных или программных компонентов системы. [ISTQB Glossary]

Модульное тестирование (оно же юнит-тестирование) используется для тестирования какого-либо одного логически выделенного и изолированного элемента системы (отдельные методы класса или простая функция, subprograms, subroutines, классы или процедуры) в коде. [Библия QA]

Компонентное тестирование - тип тестирования ПО, при котором тестирование выполняется для каждого отдельного компонента отдельно, без интеграции с другими компонентами. [Библия QA]

Интеграционное тестирование

Интеграционное тестирование (integration testing) - уровень тестирования, который фокусируется на взаимодействии между компонентами или системами

Уровни интеграционного тестирования:

- **Компонентный интеграционный уровень** (CIT - [Component Integration testing](#)): Проверяется взаимодействие между компонентами одной системы после проведения компонентного тестирования.
- **Системный интеграционный уровень** (SIT - [System Integration testing](#)): - это полное тестирование всей системы, состоящей из множества подсистем.

Подходы к интеграционному тестированию:

- **Подход Большого взрыва (Big Bang Approach)**: Все или практически все разработанные модули собираются вместе в виде законченной системы или ее основной части, и затем проводится интеграционное тестирование. Такой подход очень хорош для сохранения времени.



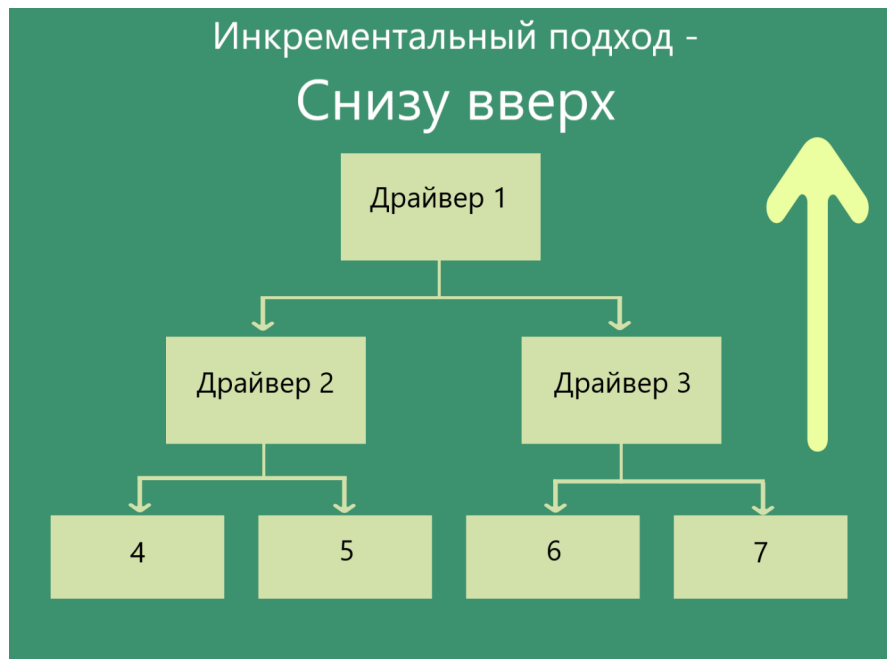
- **Инкрементальный подход (Incremental Approach)**: при таком подходе тестирование выполняется путем объединения двух или более

логически связанных модулей. Затем другие связанные модули поэтапно добавляются и тестируются для правильного функционирования. Процесс продолжается до тех пор, пока все модули не будут соединены и успешно протестированы. Осуществляется разными методами:

- **Нисходящий подход (Top-Down Approach):** Вначале тестируются все высокоуровневые модули, и постепенно один за другим добавляются низкоуровневые. Все модули более низкого уровня симулируются заглушками с аналогичной функциональностью, затем по мере готовности они заменяются реальными активными компонентами.



- **Восходящий подход (Bottom-Up Approach):** В восходящей стратегии каждый модуль на более низких уровнях последовательно тестируется с более высокоуровневыми модулями, пока не будут протестированы все модули. Для симуляции недостающих модулей используются драйвера для тестирования.



- **Гибридный/сэндвич-подход (Sandwich/Hybrid/Bi-Directional Approach):**
Представляет собой комбинацию восходящего и нисходящего подходов. Здесь целью является средний слой, в то время как драйверы заменяют верхний слой, а заглушки нижний пока компоненты этих слоев не будут разработаны;

Источник:

https://vladislavermeev.gitbook.io/ga_bible/vidy-metody-urovni-testirovaniya/integracionnoe-testirovanie-integration-testing

Системное тестирование

Системное тестирование означает тестирование всей системы в целом, оно выполняется после интеграционного тестирования, чтобы проверить, работает ли вся система целиком должным образом. В основном это тестирование типа «черный ящик», которое оценивает работу системы с точки зрения пользователя с помощью документа спецификации и оно не требует каких-либо внутренних знаний о системе, таких как дизайн или структура кода.

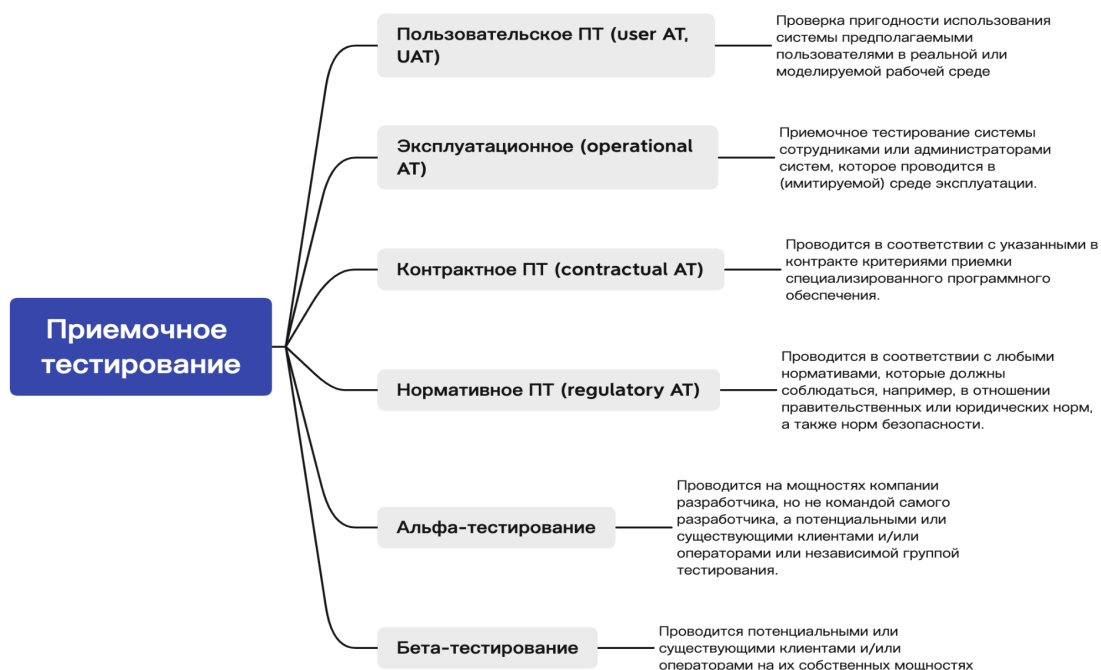
Зачем нужно системное тестирование?

- Очень важно завершить полный цикл тестирования, и ST - это этап, на котором это делается;

- ST выполняется в среде, аналогичной production environment, и, следовательно, заинтересованные стороны могут получить хорошее представление о реакции пользователя;
- Это помогает свести к минимуму устранение неполадок после развертывания и количество обращений в службу поддержки;
- На этом этапе STLC тестируются архитектура приложения и бизнес-требования. Это тестирование очень важно, и оно играет важную роль в предоставлении клиенту качественного продукта.

Приемочное тестирование

Приемочное тестирование (acceptance testing) - уровень тестирования, целью которого является определение того, принимать ли систему



1. Пользовательское приемочное тестирование (UAT, User acceptance testing).

Пользовательское приемочное тестирование проводится пользователями конечного продукта. Например, когда мы знаем, что наш продукт готов к релизу, тогда мы можем собрать некоторую группу конечных пользователей, которая его

протестируем, посмотреть, действительно ли данный продукт соответствует их ожиданиям. Они дадут нам какое-то заключение, мы исправим все критичные баги и сможем спокойно выпустить наш продукт.

2. Эксплуатационное тестирование. Обычно данное тестирование выполняется либо пользователями, либо же администратором в среде, которая имитирует реальные условия рабочего окружения. На этом этапе проводится проверка тестирования резервного копирования или, позже, аварийного восстановления системы, также его безопасность.

3. Тестирование на соответствие контракту. Данное тестирование представляет собой проверку на соответствие спецификациям какому-то другому документу, нормативному акту. Например, в стране существует ГОСТ, который определяет, как продукт должен соответствовать данному нормативному правовому акту, т.е. определяет его основные ключевые особенности. Мы должны обязательно разрабатывать продукт в полном соответствии к этому нормативному акту. Либо же один из самых известных документов - определяет требования к защите пользовательских данных. Если мы не учтем данные требования при разработке продукта, то можем получить огромные штрафы в дальнейшем.

4. Альфа-тестирование - эксплуатационное тестирование, проводится на стороне разработчика, но не в рамках нашей компании.

5. Бета-тестирование проводится на внешней стороне и без участия разработчиков. Обычно бета-тестирование проводится небольшой группой пользователей, чтобы получить их отзыв до выхода продукта на рынок, это известная практика для компьютерных игр. Т.е. заказчик выбирает некую ключевую группу пользователей и выпускает для них версию продукта, на основании их отзывов будут проводиться изменения каких-то критичных багов, либо же мы будем просто понимать, что продукт соответствует их ожиданиям и его можно уже выпускать.

По позитивности и негативности тестов

Позитивное, негативное и деструктивное тестирование

Позитивное тестирование (positive testing) - исследование приложения в ситуации, когда все действия выполняются строго по инструкции без каких бы то ни было ошибок, отклонений, ввода неверных данных и т.д.

Пример: ввод валидных значений в поле имя и пароль для успешной регистрации пользователя.

Негативное тестирование (negative testing, invalid testing) — направлено на исследование работы приложения в ситуациях, когда с ним выполняются (некорректные) операции и/или используются данные, потенциально приводящие к ошибкам (классика жанра — деление на ноль).

Пример: ввод невалидных значений в поле имя и пароль, появление сообщения об ошибке, неуспешная регистрация.

Деструктивное тестирование (destructive testing) - одна из форм негативного тестирования с целью нарушить работоспособность приложения и обнаружить точку отказа.

Пример: нагрузка приложения выше его предела, чтобы оно перестало работать

В реальном тестировании следует использовать несколько правил:

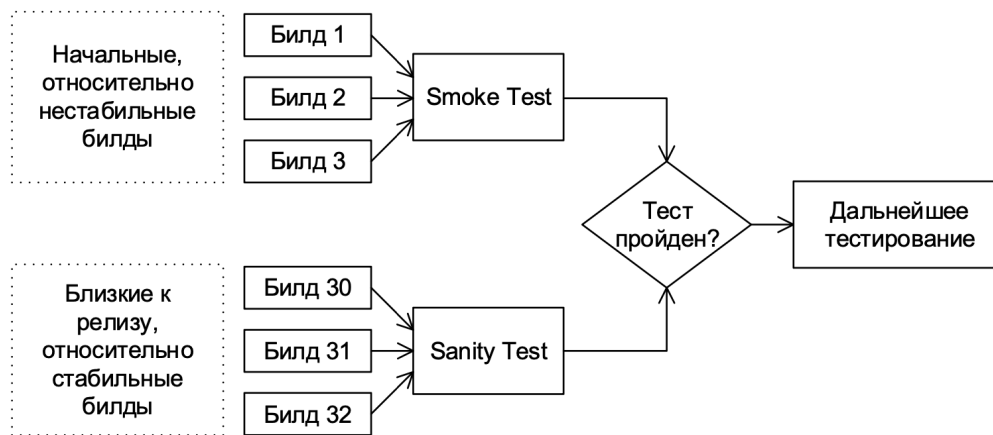
1. Любое тестирование необходимо начинать с позитивных проверок.
Наша первоочередная задача убедиться, что приложение работает в стандартных условиях и готово для использования.
2. Нельзя объединять позитивные и негативные тесты, так как это затрудняет локализацию дефектов.

Источник определений: [Святослав Куликов "Тестирование ПО. Базовый курс."](#)

По степени важности

1. **Дымовое тестирование (smoke test)** направлено на проверку самой главной, самой важной, самой ключевой функциональности, неработоспособность которой делает бессмысленной саму идею использования приложения (или иного объекта, подвергаемого дымовому тестированию).
- Очень часто можно услышать вопрос о том, чем «smoke test» отличается от «**sanity test**». В глоссарии ISTQB сказано просто: «sanity test: See

smoke test». Но некоторые авторы утверждают, что разница есть и может быть выражена следующей схемой



2. **Тестирование критического пути (critical path test)** направлено на исследование функциональности, используемой типичными пользователями в типичной повседневной деятельности.
3. **Расширенное тестирование (extended test)** направлено на исследование всей заявленной в требованиях функциональности — даже той, которая низко проранжирована по степени важности. Ещё одним направлением исследования в рамках данного тестирования являются нетипичные, маловероятные, экзотические случаи и сценарии использования функций и свойств приложения, затронутых на предыдущих уровнях.

Источник: [Святослав Куликов “Тестирование ПО. Базовый курс.”](#)

- **Smoke:** пользователь авторизовался, смог найти и добавить товар в корзину, совершил оплату и оформил доставку - в качестве примера для онлайн-магазина.

Проверка основной функциональности обычно не меняется и проводится на каждой новой промежуточной версии продукта, или билде. Иногда этот процесс называют **сертификацией билда**.

Если Smoke не проходит, то остальное тестирование останавливается и ждет доработки кода.

- **Critical path:** тестируются новые или измененные функции, которые попали в промежуточную версию продукта с точки зрения использования конечного пользователя.
- **Extended:** все остальное.

Что касается **Sanity** тестирования, то с одной точки зрения это синоним Smoke.

По цели тестирования

Ретест и регрессионное тестирование

Подтверждающее (re-testing, ретест, повторное тестирование) - тип тестирования, связанного с изменениями, которое выполняется после исправления дефекта для подтверждения того, что отказ, вызванный этим дефектом, не воспроизводится. [ISTQB Glossary]

Регрессионное тестирование (regression testing) - тип тестирования, связанного с изменениями, чтобы найти привнесенные или ранее не обнаруженные дефекты в не менявшихся областях программного обеспечения. [ISTQB Glossary]

Примечание :

Ретест подтверждает факт того, что дефект был исправлен. Он не требует оценку влияния изменений, хотя хороший тестировщик всегда потратит несколько минут на общую оценку работы приложения после такого фикса.

Регрессионное тестирование проводится после любого изменения (новая функциональность, исправление дефекта, удаление функциональности), чтобы убедиться в том, что приложение работает стабильно. Оно не включает в себя ретест, а проводится уже после него.

Нужно понимать, что на практике регресс не проводится каждый раз, когда в системе появляется изменение, а тестирует скорее "накапливающийся" эффект.

Например, две недели мы разрабатывали новые функциональности и тестировали только их. В конце этого периода (итерации) у нас запланирован релиз. И именно перед ним мы проведем одну общую регрессию для всего приложения.

Регрессионное тестирование является одной из самых главных активностей тестировщика и проводится перед каждым релизом (выпуском продукта для конечного пользователя).

Если релиз запланирован на каждую неделю, то регрессия проводится с такой же периодичностью, поэтому тесты для регрессии, как и дымные тесты очень часто автоматизируют для экономии времени.

Обычно для проведения регрессии выделяют несколько дней, код проекта "замораживают"(code freeze) и в него нельзя вносить изменения, кроме исправлений критических дефектов.

Важно понимать: какие тесты в конечном итоге попадут в регрессионный набор?

Регрессионные тесты выбираются из уже существующих тестовых наборов на основании следующих принципов:

- *Тесты, проверяющие части приложения, в которые вносились изменения*

Например, если дефект или новая функция локализованы в модуле регистрации, необходимо убедиться, что остальные компоненты этого модуля работоспособны

- *Тесты с высоким приоритетом*

Мы не можем рисковать работоспособностью самых важных функций приложения, поэтому необходимо дополнительно проверять и их

- *Тесты, которые проверяют модули с наибольшей концентрацией дефектов*

Согласно принципу скопления дефектов необходимо убедиться, что в уязвимой части приложения нет дополнительных проблем.

По степени автоматизации

Ручное тестирование (manual testing) - тестирование, в котором тест-кейсы выполняются человеком вручную без использования средств автоматизации. [Куликов]

Автоматизированное тестирование (automated testing, test automation) - набор техник, подходов и инструментальных средств, позволяющий исключить человека из выполнения некоторых задач в процессе тестирования. [Куликов]

По знанию кода

Ручные тестировщики чаще всего работают с методами черного и серого ящика.

Метод тестирования серого ящика не упоминается в ISTQB CTFL.



По функциональному и нефункциональному тестирование

Функциональное тестирование системы (functional testing) включает тесты по оценке функций, которые должна выполнять система. Данный вид тестирования можно проводить на всех уровнях.

Нефункциональное тестирование системы (non-functional testing)

выполняется для оценки таких характеристик системы и программного обеспечения, как удобство использования, производительность или безопасность. За классификацией характеристик качества программного обеспечения следует обратиться к стандарту ИСО (ISO/IEC 25010). Данный вид тестирования можно проводить на всех уровнях.

Примечание

Функциональное тестирование отвечает на вопрос "Что делает система?".

Нефункциональное тестирование отвечает на вопрос "Как система это делает?".

Инсталляционное тестирование (installation testing, installability testing) —

тестирование, направленное на выявление дефектов, влияющих на протекание стадии инсталляции (установки) приложения.

Включает в себя следующие процессы:

1. Установка ПО
2. Удаление ПО
3. Обновление ПО
4. Откат на предыдущую версию
5. Повторный запуск установки после возникновения ошибки или исправления уже возникших проблем
6. Автоматическая установка
7. Установка отдельного компонента из общего пакета программ

Тестирование удобства использования (usability testing) — тестирование, направленное на исследование того, насколько конечному пользователю понятно, как работать с продуктом (understandability, learnability, operability), а также на то, насколько ему нравится использовать продукт (attractiveness).

Что нужно тестировать:

1. Общая доступность
2. Скорость, производительность
3. Удобство навигации и интерфейс
4. Плавность

Тестирование доступности (accessibility testing, A11Y) — тестирование, направленное на исследование пригодности продукта к использованию людьми с ограниченными возможностями (слабым зрением и т.д.).

Что можно тестировать:

1. Использование вспомогательных технологий в ПО (распознавание речи, экранная клавиатура и лупа, скринридеры)
2. Возможность использовать приложение одной рукой
3. Настройки специальной цветопередачи
4. Наличие понятных инструкций и руководства пользователя

Доступность сайта можно тестировать специальными инструментами, например, WAVE, TAV, Accessibility Valet, Accessibility Developer Tools.

Тестирование безопасности (security testing) — тестирование, направленное на проверку способности приложения противостоять злонамеренным попыткам получения доступа к данным или функциям, права на доступ к которым у злоумышленника нет.

От чего нужно защищать ПО:

1. SQL-инъекции
2. XSS-инъекции
3. Перехват трафика
4. Брутфорсинг (полный перебор данных для получения доступа)

Тестирование интернационализации (internationalization testing, i18n testing, globalization testing, localizability testing) — тестирование, направленное на проверку готовности продукта к работе с использованием различных языков и с учётом различных национальных и культурных особенностей.

Тестирование локализации (localization testing, l10n) — тестирование, направленное на проверку корректности и качества адаптации продукта к

использованию на том или ином языке с учётом национальных и культурных особенностей.

Различия i18n и l10n



Т. интернационализации

Готовность к адаптации в новых локалях

Примеры:

Направление текста
Возможность использования разных кодировок
Конечный вид интерфейса



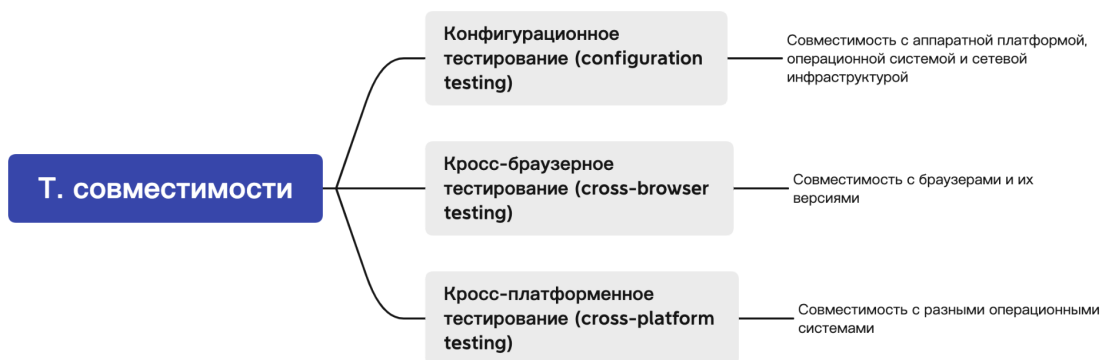
Т. локализации

Сама адаптация к локалям

Примеры:

Перевод
Валюта
Цвета
Нормативы
Единицы измерения
Формат даты и времени

Тестирование совместимости (compatibility testing, interoperability testing) — тестирование, направленное на проверку способности приложения работать в указанном окружении.



Тестирование на отказ и восстановление

- **Тестирование надёжности (reliability testing)** — тестирование способности приложения выполнять свои функции в заданных условиях на протяжении заданного времени или заданного количества операций.

- **Тестирование восстанавливаемости (recoverability testing)** — тестирование способности приложения восстанавливать свои функции и заданный уровень производительности, а также восстанавливать данные в случае возникновения критической ситуации, приводящей к временной (частичной) утрате работоспособности приложения.
- **Тестирование отказоустойчивости (failover testing)** — тестирование, заключающееся в эмуляции или реальном создании критических ситуаций с целью проверки способности приложения задействовать соответствующие механизмы, предотвращающие нарушение работоспособности, производительности и повреждения данных.

Тестирование производительности (performance testing) — исследование показателей скорости реакции приложения на внешние воздействия при различной по характеру и интенсивности нагрузке.

Подвиды:

1. Нагрузочное тестирование (load testing, capacity testing)
2. Тестирование масштабируемости (scalability testing)
3. Объёмное тестирование (volume testing)
4. Стрессовое тестирование (stress testing)
5. Конкурентное тестирование (concurrency testing)

Нагрузочное тестирование — исследование способности приложения сохранять заданные показатели качества при нагрузке в допустимых пределах и некотором превышении этих пределов (определение «запаса прочности»).

Тестирование масштабируемости — исследование способности приложения увеличивать показатели производительности в соответствии с увеличением количества доступных приложению ресурсов.

Объёмное тестирование — исследование производительности приложения при обработке различных (как правило, больших) объёмов данных.

Стрессовое тестирование — исследование поведения приложения при нештатных изменениях нагрузки, значительно превышающих расчётный уровень, или в ситуациях недоступности значительной части необходимых приложению ресурсов.

Конкурентное тестирование — исследование поведения приложения в ситуации, когда ему приходится обрабатывать большое количество

одновременно поступающих запросов, что вызывает конкуренцию между запросами за ресурсы (базу данных, память, канал передачи данных, дисковую подсистему и т.д.).

Источник: Святослав Куликов “Тестирование ПО. Базовый курс.”

Примеры для различных типов тестирования производительности от автора курса



По степени исполнения сценария

Сценарное - Тестирование на основе тест-кейсов (scripted testing, test case based testing) — формализованный подход, в котором тестирование производится на основе заранее подготовленных тест-кейсов, наборов тест-кейсов и иной документации.

Исследовательское тестирование (exploratory testing) — частично формализованный подход, в рамках которого тестировщик выполняет работу с приложением по выбранному сценарию, который, в свою очередь, дорабатывается в процессе выполнения с целью более полного исследования приложения.

Свободное (интуитивное) тестирование (ad hoc testing) — полностью неформализованный подход, в котором не предполагается использования ни тест-кейсов, ни чек-листов, ни сценариев — тестировщик полностью опирается на свой профессионализм и интуицию (experience-based testing) для спонтанного выполнения с приложением действий, которые, как он считает, могут обнаружить ошибку.

Примеры:

1. **Т. на основе тест-кейсов:** у вас есть четкие требования, вы написали тестовую документацию с конкретными шагами и проверками и приступаете к тестированию.
2. **Исследовательское Т.:** у вас есть требования или общее понимание работы продукта, вы набросали первичные проверки, которые будете дорабатывать по ходу тестирования.
3. **Свободное Т.:** у вас могут быть требования или полностью отсутствовать, вы не создаете набросок тестирования или официальную документацию, а тестируете приложение основываясь только на своем опыте.

Частные случаи: **monkey testing, gorilla testing**, о которых вы можете почитать дополнительно.

По запуску кода

Статическое тестирование (static testing) - тестирование, которое не предполагает выполнение тестируемого компонента или системы

Примеры: тестирование документации, прототипов, кода (в рамках code review), тестовых данных. Есть специальные техники для статического анализа.

Динамическое тестирование (dynamic testing) - тестирование, проводимое во время выполнения тестируемого элемента.

Пример: проверка реального поведения приложения при запуске кода на разных уровнях тестирования

Вопросы по лекции

1. Как работает методика тестирования черного ящика?
2. Какие тесты запускаются раньше по степени позитивности?
3. Верно ли утверждение? *В результате модульного тестирования несколько модулей тестируются вместе.*
4. Какие тесты следует добавить в регрессионный набор?
5. Как работает методика тестирования белого ящика?
6. Верно ли утверждение? *Ручные тестировщики чаще всего работают с методом белого ящика.*
7. Какие типы тестирования следует отнести к нефункциональным?
 - Нагрузочное тестирование
 - Тестирование локализации
 - Дымное тестирование
 - Тестирование безопасности
 - Тестирование совместимости
8. Тестирование производительности включает в себя следующие подвиды:
 - Тестирование совместимости
 - Объёмное тестирование
 - Тестирование безопасности
 - Нагрузочное тестирование
 - Стрессовое тестирование